Gobelo Corpus Builder Toolkit

gcbt v4.0

User Manual

Multi-Format Text Extraction, Cleaning & Segmentation for Zambian Bantu Language Corpus Research

Gobelo Grammar Toolkit (GGT) · MIT License

1. Overview

The Gobelo Corpus Builder Toolkit (gcbt) is a production-ready Python pipeline that ingests heterogeneous document collections, cleans and normalises text for Bantu linguistic contexts, segments it into one sentence per line, and exports structured output ready for NLP model training or corpus linguistics research.

1.1 Supported languages

gcbt ships with profiles for all seven official Zambian regional languages. Additional languages can be added without modifying any Python — see Section 6.

ISO code	Language	Guthrie zone	Chapter word
toi	ChiTonga	M.64	Matalikilo / Mutatwe
bem	ChiBemba	M.42	Mutwalo / Mutwe
nya	ChiNyanja / Chichewa	N.31	Mutu
loz	SiLozi	K.21	Kauhanyo
lun	ciLunda	L.52	Mutwalo
lue	ciLuvale	K.14	Mutwalo
kqn	ciKaonde	L.41	Mutwalo
und	Universal (fallback)	—	Chapter / Part / …
auto	Auto-detect from path	—	(inherits detected profile)

1.2 Supported file formats

Extension(s)	Format	Notes
.pdf	PDF	pdfplumber (default) · PyMuPDF · PyPDF2 fallback
.docx / .doc	Word document	python-docx
.epub	EPUB e-book	ebooklib + BeautifulSoup
.html / .htm	HTML	BeautifulSoup; scripts and styles stripped
.md / .markdown	Markdown	Rendered to HTML then extracted
.txt / .text	Plain text	Encoding auto-detected via chardet
.srt	SubRip subtitles	Timecodes, sequence numbers and HTML tags stripped
.csv	CSV	All string-typed columns extracted
.xml	XML	Tag-stripped via BeautifulSoup

1.3 Processing pipeline

Each file passes through four sequential stages:

1. **Extract** — format-specific text extraction (Stage 1)
2. **Clean** — Unicode normalisation, OCR correction, URL/email removal, chapter-heading stripping, single-char filtering (Stage 2)
3. **Segment** — sentence splitting using NLTK (with language-aware abbreviation sets) or spaCy (Stage 3)
4. **Export** — .txt (one sentence per line), .json (metadata + sentences), or .cleaned.txt (Stage 4)

2. Installation

2.1 Requirements

- Python 3.9 or later
- PyYAML (required for the two-layer config system)
- NLTK (recommended for sentence segmentation)
- All other dependencies are optional — gcbt degrades gracefully when a library is absent

2.2 Install dependencies

```
pip install PyYAML nltk
pip install pdfplumber PyPDF2 python-docx ebooklib beautifulsoup4 markdown
pip install chardet rich psutil
# Optional: spaCy for advanced segmentation
pip install spacy && python -m spacy download en_core_web_sm
# Optional: PyMuPDF for margin-aware PDF extraction
pip install pymupdf
```

2.3 Project layout

```
your_project/
  gcbt_v40.py          ← main application
  corpus_config.yaml   ← two-layer configuration (edit this)
  languages/           ← GGT grammar YAML files (one per language)
    chitonga.yaml
    chibemba.yaml
    ...
  logs/                ← auto-created; one log file per run
```

> **NOTE:** gcbt looks for corpus_config.yaml in the current working directory by default. Use --corpus-config to point elsewhere.

3. Quick Start

3.1 Organise your corpus

Place your source files in a directory tree whose top-level folders name the subject category. gcbt mirrors this structure exactly in the output.

```
input/
  chitonga/
    literature/   novel1.pdf  poem.docx
    religious/    bible.txt
    academic/     paper.pdf
```

3.2 Run in batch mode

```
python gcbt_v40.py -i input/ -o output/
```

gcbt auto-detects the language from directory names (e.g. a folder named chitonga/ → toi profile). Output mirrors the input tree:

```
output/
  chitonga/
    literature/
      novel1.txt    ← one sentence per line
      novel1.json   ← metadata + sentences array
```

3.3 Run interactively

Launch without arguments to enter the guided menu:

```
python gcbt_v40.py
```

You will be prompted for: corpus config path, input directory, output directory, language mode, and processing preset.

4. Command-Line Reference

4.1 Core arguments

Flag	Short	Default	Description
--input	-i	(required)	Directory containing source files
--output	-o	(required)	Directory for output files
--language	-l	auto	ISO code, 'und' (universal), or 'auto' (path-detect)
--corpus-config		corpus_config.yaml	Path to the two-layer YAML config

Flag	Short	Default	Description
--preset		standard	minimal · standard · aggressive · pristine
--formats		txt,json	Output formats: comma-separated txt / json / cleaned
--config	-c		Load pipeline config from a JSON file
--save-config			Save current pipeline config to a JSON file
--no-recursive		off	Process top-level directory only (disable recursion)
--log-level		INFO	DEBUG · INFO · WARNING · ERROR

4.2 Cleaning flags

Flag	Default	Effect when set
--no-remove-urls	remove enabled	Keep URLs in output text
--no-remove-emails	remove enabled	Keep email addresses in output text
--preserve-bible	strip enabled	Keep scripture verse references (e.g. Gen 1:1)
--no-process-ocr	OCR enabled	Skip OCR glyph corrections
--no-filter-single-chars	filter enabled	Keep all single-character tokens
--min-sentence-length	3	Discard sentences shorter than N characters
--max-sentence-length	1000	Split sentences longer than N characters
--no-nltk	NLTK enabled	Use the fallback character-scan segmenter
--pdf-method	pdfplumber	pdfplumber · pypdf2 · pymupdf
--max-file-size	100	Skip files larger than N MB
--max-memory	250	Trigger GC when process exceeds N MB

4.3 Processing presets

Presets are shortcuts for common cleaning intensity levels. A preset replaces the full CleaningConfig — individual CLI flags cannot partially override a preset; use the JSON config for fine-grained control.

Preset	Use case	Key differences from standard
minimal	Raw extraction only	OCR off · single-char filter off
standard	General purpose (default)	All defaults from corpus_config.yaml
aggressive	Maximum noise removal	Strict single-char filter · min line length 5
pristine	Research-grade output	NFKD normalisation · junk-consonant removal · citation protection · spaCy if available

4.4 Example commands

Auto-detect language from directory names

```
python gcbt_v40.py -i corpus/ -o processed/
```

Force a specific language

```
python gcbt_v40.py -i bemba_docs/ -o out/ --language bem
```

Use a custom corpus config file

```
python gcbt_v40.py -i docs/ -o out/ --corpus-config /data/my_corpus_config.yaml
```

Aggressive preset, Nyanja, TXT output only

```
python gcbt_v40.py -i nyanja/ -o out/ --preset aggressive --language nya --
formats txt
```

Preserve bible text, debug logging

```
python gcbt_v40.py -i religious/ -o out/ --preserve-bible --log-level DEBUG
```

Save and reuse a pipeline config

```
python gcbt_v40.py -i corpus/ -o out/ --save-config my_run.json
python gcbt_v40.py -i corpus2/ -o out2/ --config my_run.json
```

5. Configuration System

5.1 Two-layer architecture

gcbt uses two separate YAML sources that are merged at startup into a LanguageProfile for each language. This separation keeps linguistic data and corpus-processing settings in their proper places.

Layer	File	Contains
1 — Linguistic	languages/<iso>.yaml (GGT grammar file)	ISO code · language name · Guthrie zone · phonology · orthography · special characters
2 — Pipeline	corpus_config.yaml (this project)	Chapter words · abbreviations · OCR corrections · single-char morphemes · strip patterns · pipeline toggles

> **KEY RULE:** Never put linguistic data in corpus_config.yaml, and never put corpus-processing settings in the GGT YAML. The GGT grammar files are the authoritative source for all linguistic information.

5.2 corpus_config.yaml structure

The file has two top-level sections:

```
global:          # baseline applied to every language
  verse_pattern: '...'
```

```
  punctuation_map: {...}
  chapter_words: [Chapter, Part, ...]
  book_abbreviations: [Dr, Mr, ...]
  ocr_corrections: {'rn': 'm', ...}
  valid_single_chars: [a, e, i, o, u, n, m]
  extra_special_chars: []
  strip_patterns: []
  # Pipeline toggles (true/false)
  remove_urls: true
  filter_single_chars: true
  # ... (see Section 5.3)

languages:
  toi:                            # ISO 639-3 code
    ggt_yaml: languages/chitonga.yaml  # required
    path_keywords: [toi, chitonga, tonga]
    chapter_words: [Matalikilo, Mutatwe]
    book_abbreviations: [Mw, Ba, Sh]
    valid_single_chars: [w, b, y, k]
    extra_special_chars: [ng, ny]
    ocr_corrections:
      'cl': 'ci'
      '0':  'o'
```

5.3 Merging rules

Field	Rule	Effect
chapter_words	ADDITIVE	Language list ∪ global list
book_abbreviations	ADDITIVE	Language set ∪ global set
valid_single_chars	ADDITIVE	Language set ∪ global set
extra_special_chars	ADDITIVE (ordered)	Global list + language list; order preserved
strip_patterns	ADDITIVE (ordered)	Global list + language list; order preserved
ocr_corrections	KEY-BY-KEY OVERRIDE	Language entry wins for any shared key; global fills the rest
punctuation_map	GLOBAL ONLY	Shared across all languages; not overridable per language
Pipeline toggles	GLOBAL ONLY	remove_urls, filter_single_chars, fix_hyphenation, etc.

5.4 Pipeline toggles (global section)

Key	Default	Description
remove_urls	true	Remove http/https/www URLs from text
remove_emails	true	Remove email addresses
preserve_bible_text	false	When true, verse references are NOT stripped even if remove_bible_references is true
filter_single_chars	true	Remove single-character tokens not in valid_single_chars
fix_hyphenation	true	Rejoin words split by a hyphen across a line break
normalize_whitespace	true	Collapse runs of spaces and tabs
normalize_unicode	true	Apply NFC Unicode normalisation (NFKD in pristine mode)
normalize_punctuation	true	Replace typographic characters using punctuation_map
process_ocr_artifacts	true	Apply ocr_corrections substitutions
protect_citations	false	Shield 'et al.' and similar from sentence splitting
min_sentence_length	3	Discard sentences shorter than this many characters
max_sentence_length	1000	Split or truncate sentences exceeding this length

6. Adding a New Language

A linguist or educator can add a new language without touching any Python code. The process requires only two YAML edits.

Step 1 — Create the GGT grammar YAML

Place a grammar file in the languages/ directory. At minimum it should contain:

```yaml
# languages/nsenga.yaml
metadata:
  language:
    iso_code: nse
    name: Nsenga
    guthrie: N.41
phonology:
  vowels: aeiou
  consonants: bcdfghjklmnpqrstvwxyz
```

If a GGT grammar file is not yet available, omit the ggt_yaml key — gcbt will derive identity from the corpus_config.yaml block alone.

Step 2 — Add a block to corpus_config.yaml

Under the languages: section, add a block using the ISO code as the key:

```
languages:
  ... existing languages ...

  nse:
    ggt_yaml: languages/nsenga.yaml   # path to step-1 file
    display_name: 'Nsenga'            # optional human-readable override
    path_keywords: [nse, nsenga]      # directory name hints

    chapter_words:
      - Mutu                          # language-specific heading word

    book_abbreviations:
      - Amai
      - Bambo

    valid_single_chars:
      - a                             # only list chars BEYOND the global set

    extra_special_chars:
      - ng
      - ny

    ocr_corrections:
      'iii': 'ili'                    # language-specific OCR substitutions
```

Step 3 — Restart gcbt

That is all. No Python changes. On next startup gcbt discovers the new block, merges it with the global defaults, and exposes it as --language nse on the CLI.

> **TIP:** If you omit ggt_yaml, the new language still works — it inherits all global defaults and uses only the fields you specify in the language block.

7. Output Format

7.1 Directory structure

Output mirrors the input directory tree exactly. For each source file gcbt creates a .txt and/or .json file in the corresponding output subdirectory.

```
input/chitonga/literature/novel1.pdf
  →  output/chitonga/literature/novel1.txt
  →  output/chitonga/literature/novel1.json
```

7.2 .txt output

One sentence per line, UTF-8 encoded. This is the primary training-data format.

```
Ndakasiyila amapenda.
Bulelo lwamusuna mwaya.
Bantu bapita kunyika.
```

7.3 .json output

Metadata block plus the full sentences array, suitable for downstream analytics:

```
{
  "filename": "novel1.pdf",
  "language_iso": "toi",
  "language_name": "ChiTonga",
  "subject_category": "literature",
  "original_format": ".pdf",
  "processing_timestamp": "2025-06-01T14:32:00",
  "statistics": {
    "sentence_count": 842,
    "word_count": 7213,
    "character_count": 38420,
    "avg_words_per_sentence": 8.57
  },
  "sentences": ["Ndakasiyila amapenda.", ...]
}
```

7.4 corpus_manifest.json

Written to the output root after each run. Groups all output file paths by language ISO code, making it easy for downstream pipelines to find files for a specific language:

```
{
  "generated": "2025-06-01T14:35:00",
  "gcbt_version": "4.0",
  "config_source": "/data/corpus_config.yaml",
  "languages": {
    "toi": ["output/chitonga/literature/novel1.txt", ...],
    "bem": ["output/chibemba/bible.txt", ...]
  },
  "stats": {"toi": 12, "bem": 8}
}
```

7.5 processing_stats.json

Written to the output root. Contains pipeline performance metrics: files processed, success/failure rate, sentence/word counts, reduction percentage, memory warnings, and per-stage error counts.

8. Python API

gcbt can be embedded in a larger pipeline by importing its classes directly.

8.1 Basic usage

```
from gcbt_v40 import CorpusBuilder, UnifiedConfig
from pathlib import Path

config   = UnifiedConfig(language='toi')
pipeline = CorpusBuilder(config)
pipeline.process_directory(Path('input/'), Path('output/'))
print(pipeline.get_stats())
```

8.2 Custom config

```
from gcbt_v40 import UnifiedConfig, CleaningConfig, SegmentationConfig

config = UnifiedConfig(
    language='bem',
    corpus_config_file='/data/my_corpus_config.yaml',
    cleaning=CleaningConfig(
        remove_bible_references=True,
        filter_single_chars=True,
    ),
    segmentation=SegmentationConfig(
        min_sentence_length=5,
    ),
)
```

8.3 Using a preset

```
config = UnifiedConfig.preset_aggressive(language='nya')
# or: preset_minimal, preset_standard, preset_pristine
```

8.4 Hot-reloading language profiles

```
from gcbt_v40 import reload_profiles
reload_profiles('/data/updated_corpus_config.yaml')
```

This updates the module-level LANGUAGE_PROFILES registry in place. Subsequent CorpusBuilder instances will use the new profiles. Existing instances are unaffected.

8.5 Backward-compatibility aliases

The following aliases are preserved for scripts written against earlier versions:

Alias	Resolves to
CorpusProcessingPipeline	CorpusBuilder
GobeloCorpusBuilder	CorpusBuilder
ChitongaTextCleaner	TextCleaner
fix_ocr_errors (method)	TextCleaner.fix_ocr_corrections()

9. Language Auto-Detection

When --language auto is used (the default), gcbt inspects the full file path — both the directory names and the filename — for substrings matching each language profile's path_keywords list.

9.1 Detection priority

5. All parent directory names are scanned, deepest first.
6. The filename itself is scanned.
7. If no keyword matches, the universal 'und' profile is used.

9.2 Examples

Path	Detected language
/corpus/chitonga/literature/novel.pdf	toi (keyword: chitonga)
/corpus/chibemba/bible.txt	bem (keyword: chibemba)
/corpus/silozi/proverbs.txt	loz (keyword: silozi)
/data/mixed/document.pdf	und (no keyword match)

> **TIP:** Name your corpus directories after the language (chitonga/, silozi/, etc.) and gcbt will never need a --language flag.

10. Troubleshooting

Symptom	Likely cause	Fix
ConfigError: corpus_config.yaml not found	Run from wrong directory or missing file	Use --corpus-config /full/path/to/corpus_config.yaml
PyYAML not installed	PyYAML missing	pip install PyYAML
All files detected as 'und'	path_keywords not set for the language	Add path_keywords to the language block in corpus_config.yaml
Output sentences contain 'Gen 1:1' references	Bible references not being stripped	Set remove_bible_references: true in CleaningConfig or use --preserve-

Symptom	Likely cause	Fix
		bible to intentionally keep them
SRT subtitles include timecodes in output	File processed by wrong extractor	Check that SRTProcessor is registered before TXTProcessor in ProcessorFactory (fixed in v4.0)
Empty output for a file	File too short after cleaning, or extraction failed	Check logs/ directory; try --log-level DEBUG; try --preset minimal
Memory warnings in log	Large files exceed --max-memory limit	Increase --max-memory or reduce --max-file-size
'no' missing from abbreviations	Unquoted 'no' in YAML parsed as boolean	Always quote it: - "no" in corpus_config.yaml

11. Version History

Version	Key changes
v4.0 (current)	Two-layer YAML architecture · CorpusConfigLoader · LanguageProfile has no hardcoded data · all corpus_config.yaml keys renamed to correct names (book_abbreviations, ocr_corrections) · new fields: extra_special_chars, strip_patterns, display_name, path_keywords · SRTProcessor (timecode stripping) · TextPatterns.verse_reference() live method · clean() reads config.cleaning.* exclusively · UnifiedConfig syncs toggles from YAML
v3.0	CorpusBuilder · LanguageProfile dataclass · 7-language registry hardcoded in Python · auto-detect from path · universal Bible verse regex · language-aware abbreviation sets
v2.2	ChitongaCleaner · CorpusProcessingPipeline · Bible book list filter · Chitonga-only